

Code Impact & Autonomous Repair

A multi-agent system that reads a GitHub repository, builds a directional knowledge graph of it, finds real defects, reasons about how a change travels across files, generates a minimal surgical patch, validates it behind four gates, and opens a pull request with the evidence attached.

Python 3.11

Streamlit

NetworkX

Claude (Sonnet + Haiku)

ChromaDB

Bandit

pytest

GitHub API

SCOPE OF THIS REPORT

Derived entirely from the current source tree. 29 first-party Python modules, ~4,900 lines, 11 automated tests.

SYSTEM SHAPE

10 specialised agents coordinated by 1 orchestrator across 5 pipeline phases, with 4 validation gates and a 5-signal confidence model.

DESIGN PRINCIPLE

Never trust generated code. Every fix must survive syntax, test, regression and security checks before a human is asked to look at it.

HONEST BY DEFAULT

When a tool is missing, the system reports the missing signal instead of quietly awarding confidence for it.

The thesis in one line: understanding *where a change travels* is what separates a code generator from a repair system.

What this report covers

1	Executive summary	3
2	Problem statement and motivation	3
3	System context and technology stack	4
4	Layered architecture	5
4.1	Agent architecture: the complete flow	6
5	The five-phase pipeline	7
6	The knowledge graph: construction	8
7	Directional impact analysis and blast radius	9
8	Agent catalogue	10
9	Detection strategy: scanners plus a two-stage LLM hunt	11
10	Surgical patching: anchored SEARCH/REPLACE	12
11	The four validation gates	13
12	Confidence scoring and publication policy	14
13	Semantic memory and cost control	15
14	User interface architecture	16
15	Engineering quality: tests and defects fixed	17
16	Limitations, trade-offs and future work	18
17	Conclusion	18

Figures

Figure	Title	Page
Fig 1	System context — actors, boundaries and external services	4
Fig 2	Layered architecture of the application	5
Fig 2a	Agent architecture — all ten agents, their phases and every exit path	6
Fig 3	End-to-end pipeline, Phase 0 through Phase 5	7
Fig 4	Knowledge graph schema — node and edge types	8
Fig 5	Directional impact: why edges are traversed in reverse	9
Fig 6	Two-stage detection funnel (Haiku triage → Sonnet confirmation)	11
Fig 7	Anchored patch application and the atomicity rule	12
Fig 8	The four-gate validation funnel	13
Fig 9	Confidence composition and publication decision	14
Fig 10	Streamlit concurrency model — background thread and event queue	16

A note on sourcing. Every number, threshold and behaviour in this report was read out of the code rather than from earlier documentation. Where older documentation disagrees with the implementation, the implementation is treated as the truth.

SECTION 1

Executive summary

The system takes a GitHub repository URL and returns pull requests. Between those two points it performs structural analysis, defect detection, cross-file impact reasoning, patch generation, and multi-gate validation — and it refuses to publish anything it could not verify.

10

Agents

Each owns one narrow responsibility. An eleventh module, the orchestrator, owns the sequencing.

5

Phases

Discovery, investigation, planning, fix-and-validate, publication — after a Phase 0 shallow clone.

4

Gates

Syntax, tests plus static analysis, regression across the blast radius, and a differential security scan.

What makes it different from a code generator

A conventional LLM coding tool reads one file and rewrites it. This system builds a graph of the whole repository first, so it can answer a question single-file tools cannot: *if I change this line, what else breaks?* That answer feeds three separate decisions — which files to show the model as editable context, which tests to run after the patch, and how much confidence the resulting fix deserves.

The five design commitments

Commitment	How it is enforced in code
Minimal diffs	The model may only emit anchored SEARCH/REPLACE blocks. Whole-file regeneration is impossible by construction, and a patch touching more than 60% of a file (over 25 lines) is rejected as a likely rewrite.
Atomicity	If any single edit in a patch set fails to apply, the entire patch is discarded and regenerated with the failure fed back as context.
Verified, not asserted	Confidence is composed from five observed signals. A signal that could not be measured contributes zero — it is never assumed.
Honest degradation	ChromaDB, bandit, ESLint, pip-audit and a C/C++ compiler are all optional. Each has an explicit availability guard; the pipeline completes without them, with the missing signal surfaced to the user.
Human in the loop	The system opens pull requests; it never merges. Anything below the 70% threshold, and anything touching a critical path, opens as a draft.

SECTION 2

Problem statement and motivation

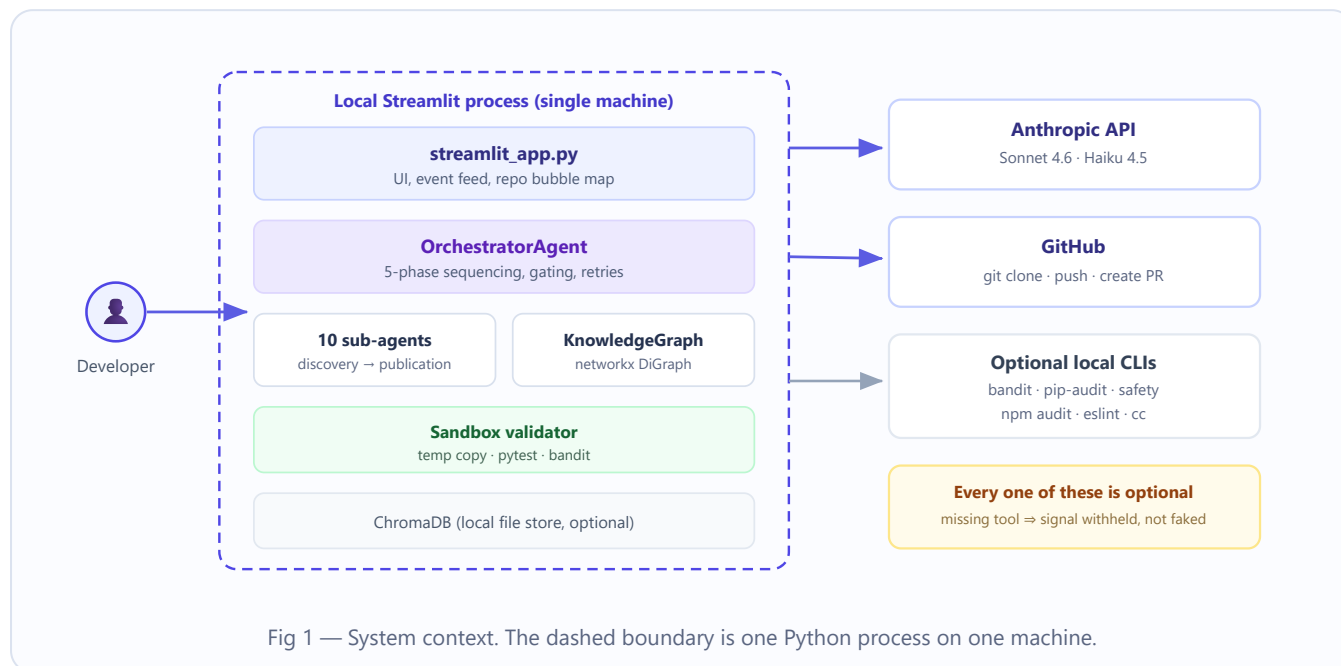
Automated bug-fixing tools generally fail in one of three ways. They **hallucinate context**, rewriting code they were never shown. They **fix symptoms**, patching the line where a crash surfaced rather than the function that produced the bad value. And they **cannot prove anything**, handing a developer a diff with no evidence that it works or that it broke nothing else.

This project targets all three:

- **Against hallucination** — the patch format requires the model to quote existing code verbatim as an anchor. Text it invents will not match, and the edit is rejected with a diagnostic rather than applied.
- **Against symptom-fixing** — the code generator is shown the epicenter file *and* its graph neighbours, and is explicitly instructed to edit the neighbour when the root cause lives there.
- **Against unverifiable output** — every patch is applied to a disposable copy and run through four gates before a PR body is written. The PR carries the gate results, the blast-radius summary and the confidence breakdown.

System context and technology stack

The application is a single Streamlit process. There is no separate API server, no database server, and no message broker — a deliberate choice explained in Section 14. Everything outside the dashed boundary below is an external dependency the system calls out to.

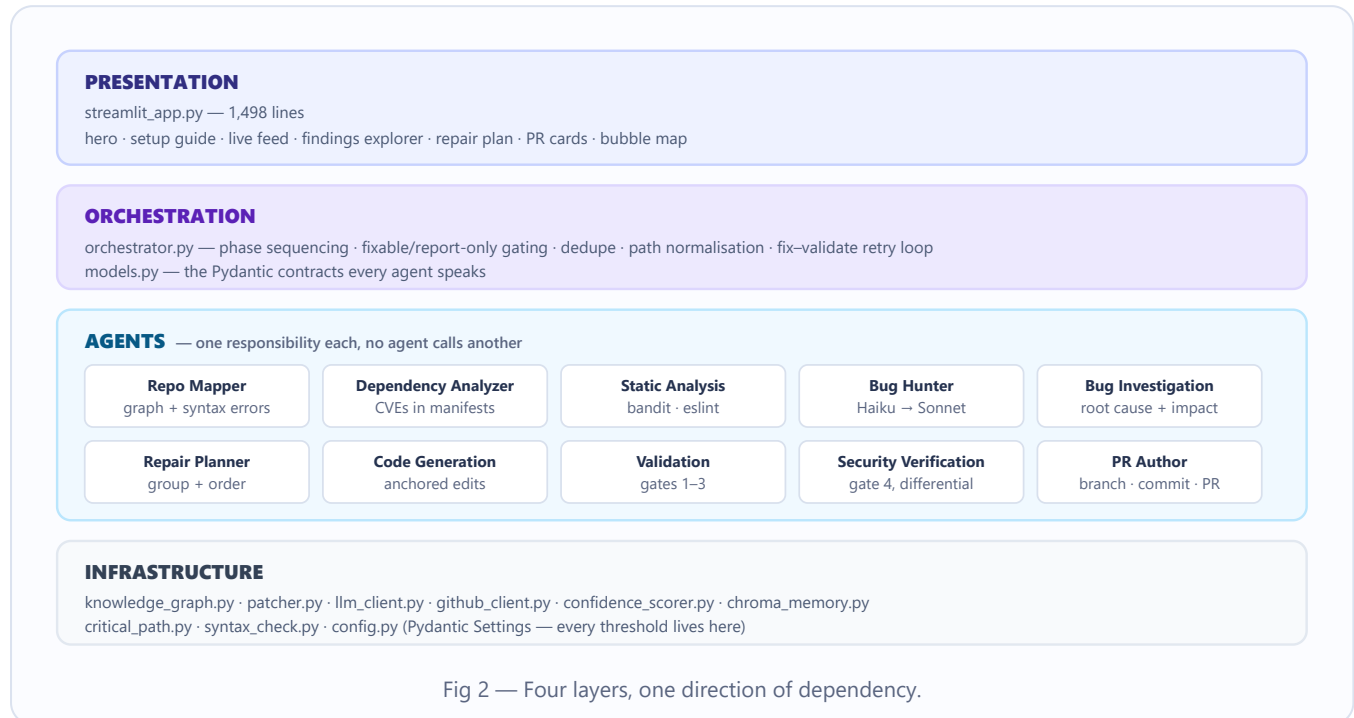


Technology choices and why

Technology	Used for	Reason it was chosen
NetworkX	The repository knowledge graph	An in-memory <code>DiGraph</code> is sufficient — the graph is rebuilt per run and never outlives the process, so a graph database would add operational weight for no gain.
Python <code>ast</code>	Parsing source structure	Standard-library, exact, and the same parser the interpreter uses — so a file that fails to parse is a genuine syntax error worth reporting as a finding.
Claude, two tiers	Detection and repair	Haiku triages every file cheaply; Sonnet is spent only on files that already have candidates, and on the reasoning and code-generation steps.
Streamlit	The entire front end	One process, no separate frontend build, no CORS, no deployment story. The pipeline runs on a background thread and streams events through a <code>queue.Queue</code> .
ChromaDB	Semantic memory of past fixes	Embedded and file-backed; needs no server. Its import is wrapped in a try/except because it pulls native extensions that some Windows security policies block.
Pydantic	Every object crossing an agent boundary	Agents exchange typed models, not dictionaries, so a malformed handoff fails loudly at the boundary rather than three phases later.

Layered architecture

The codebase separates into four layers with a strict dependency direction: presentation depends on orchestration, which depends on agents, which depend on shared infrastructure. Nothing points back up. This is what allows the pipeline to be tested and reasoned about without Streamlit.



Why agents never call each other

Each agent exposes one `async run()` or `verify()` method and returns a typed model. All sequencing lives in the orchestrator. The payoff is that any agent can be unit-tested, reordered or replaced without touching the others.

Why configuration is centralised

`config.py` is a single Pydantic `Settings` class holding every threshold, token budget, model ID and feature flag. Tuning behaviour never requires editing agent logic, and the defaults are visible in one place during a code review.

Agent architecture: the complete flow

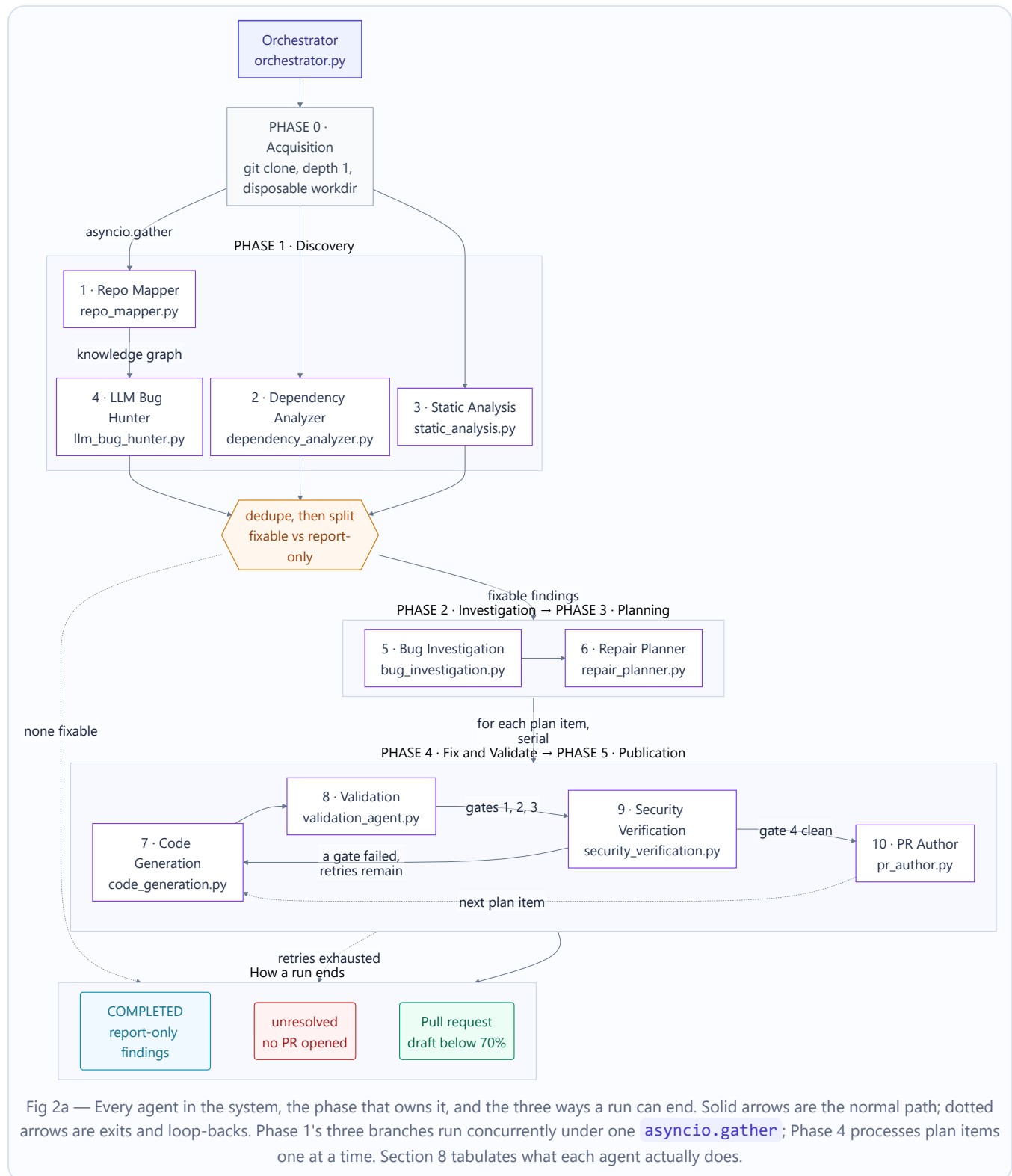
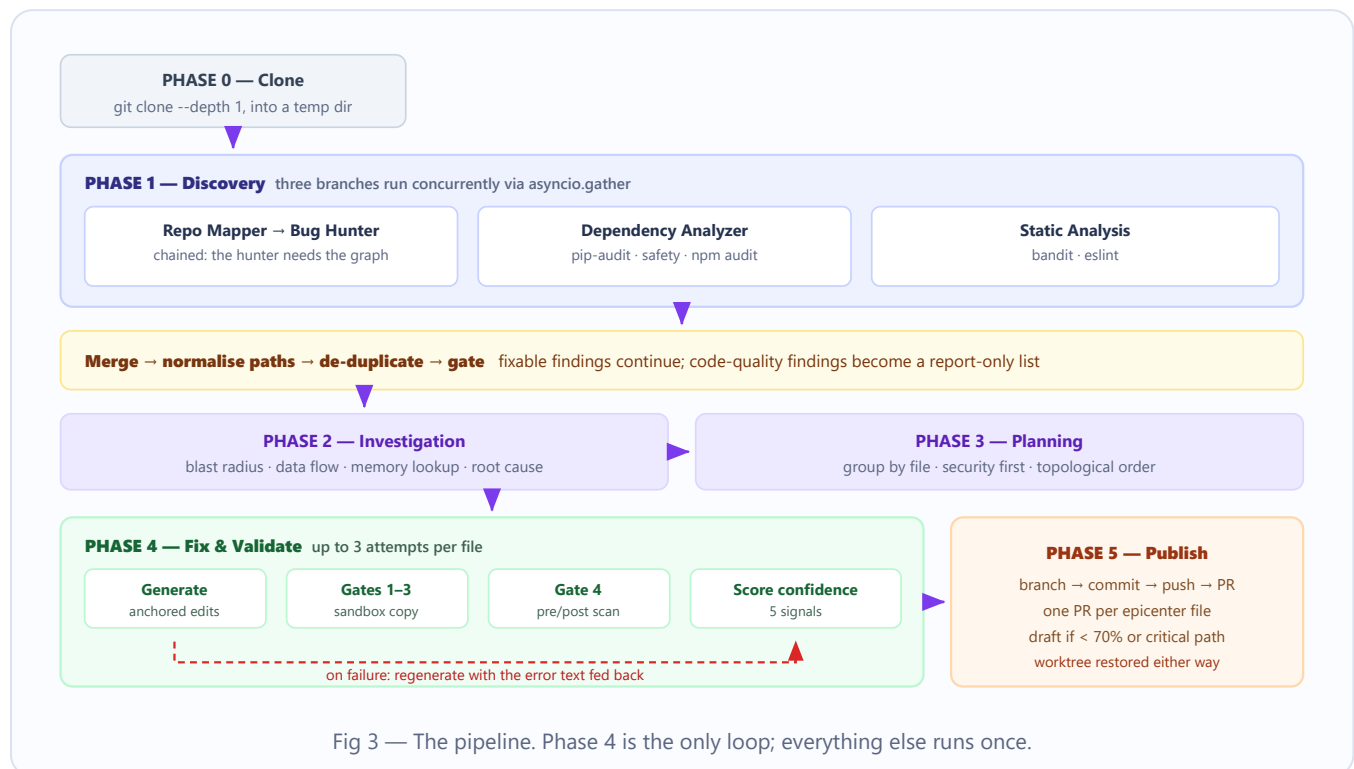


Fig 2a — Every agent in the system, the phase that owns it, and the three ways a run can end. Solid arrows are the normal path; dotted arrows are exits and loop-backs. Phase 1's three branches run concurrently under one `asyncio.gather`; Phase 4 processes plan items one at a time. Section 8 tabulates what each agent actually does.

SECTION 5

The five-phase pipeline

A run is a linear sequence of phases with one loop inside it. The `JobStatus` enum mirrors these phases exactly, which is what lets the UI render a live stepper without inventing its own state machine.



Two decisions worth defending

Why Phase 1 is concurrent but Phase 4 is serial

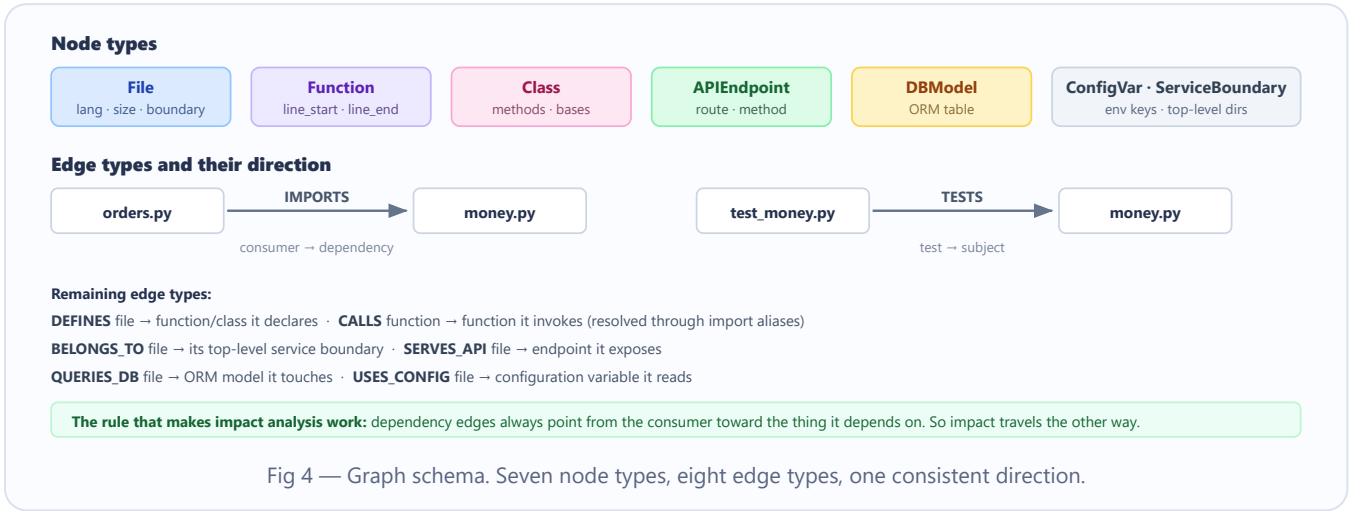
Discovery tasks are independent and mostly I/O-bound, so `asyncio.gather` is free speed. Phase 4 is deliberately serial: fixes are applied to one shared worktree, each validation run spawns subprocesses, and the fix-order signal is only meaningful if there is an order. Parallelising it would need one worktree per fix.

Why one PR per file, not per finding

Findings cluster. Three issues in the same module usually share a root cause and a test, and three separate PRs touching the same file would conflict with each other. The planner groups findings by epicenter file, so a reviewer sees one coherent change per PR.

The knowledge graph: construction

The graph is the backbone of every cross-file claim the system makes. It is a `networkx.DiGraph` built in four passes, keyed on repository-relative POSIX paths so that Windows separators and absolute paths can never produce two nodes for one file.



The four construction passes

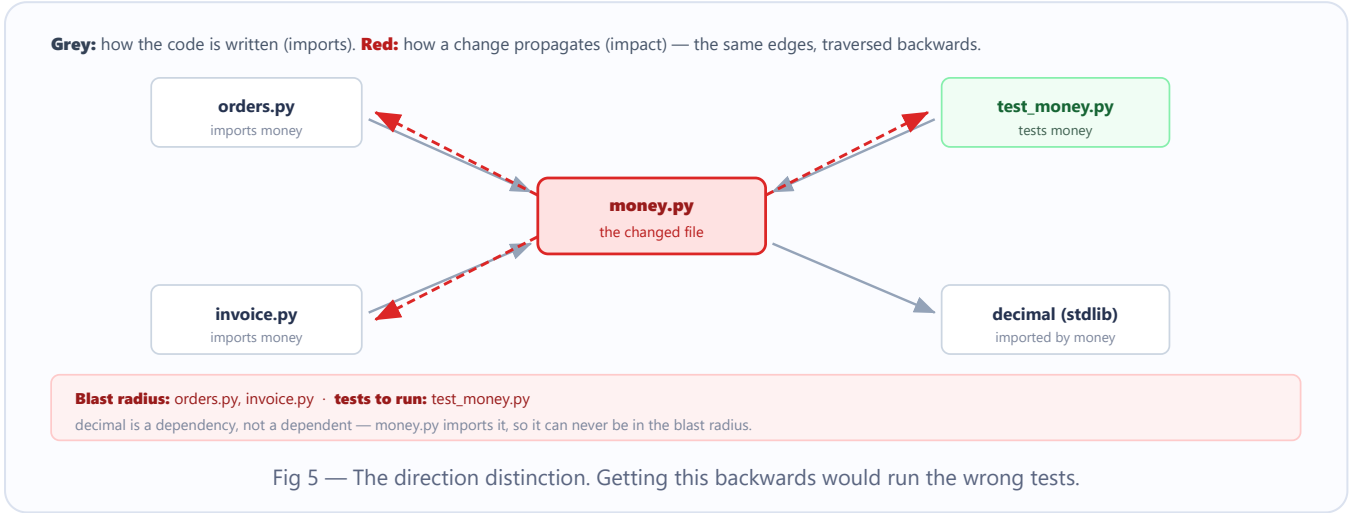
Pass	What it adds
1 — Structure os.walk	Walks the tree, skipping hidden directories plus <code>venv</code> , <code>node_modules</code> , <code>__pycache__</code> , <code>build</code> , <code>dist</code> . Creates a <code>File</code> node per file with language and size, a <code>ServiceBoundary</code> node per top-level directory, and <code>BELONGS_TO</code> edges. File size is read inside a guarded <code>stat()</code> so a broken symlink skips one file instead of aborting the scan.
2 — AST ast.parse	Parses every Python file. Extracts functions, classes, imports and call sites; resolves dotted module names to real files through a module index so <code>IMPORTS</code> becomes a genuine file-to-file edge rather than a string. Records import aliases, which is what allows <code>CALLS</code> to be resolved across files. A file that fails to parse is appended to <code>syntax_errors</code> with the parser's exact line and message.
3 — Framework regex	Recognises FastAPI, Flask and Django route declarations, SQLAlchemy/Django ORM queries, and <code>os.environ</code> / settings access — producing <code>APIEndpoint</code> , <code>DBModel</code> and <code>ConfigVar</code> nodes. Regex is deliberate here: these are decorator and attribute patterns, and a miss costs one edge rather than correctness.
4 — Dynamic	Marks test files and attaches <code>TESTS</code> edges by matching <code>test_<stem>.py</code> and <code><stem>_test.py</code> against source files. This is what later lets validation run the <i>relevant</i> tests instead of the whole suite.

Why a syntax error becomes a finding. Most analysis tools skip files they cannot parse. Here, an unparseable Python file is reported as a HIGH-severity functional bug with the real line number, because the interpreter would fail on it too. It costs nothing — the parser already produced the diagnostic — and it turns a silent gap into a result.

Directional impact analysis and blast radius

This is the part of the system worth understanding first. Everything else consumes its output.

Import and call edges point from the consumer to the thing it depends on. That is the natural reading direction of the source code. But *impact* flows the opposite way: if `orders.py` imports `money.py`, then changing `money.py` is what puts `orders.py` at risk — not the reverse. So `analyze_change()` starts at the changed file and performs a breadth-first walk over *predecessors*, collecting the paths it travelled rather than just the set of nodes it reached.



What `analyze_change()` returns, and who uses it

Field	Consumed by
<code>affected_files</code>	The confidence-relevant regression scope, and the "narrow / moderate / wide" summary printed in the PR body.
<code>related_tests</code>	Gate 3, to decide which test modules constitute a regression check for this change.
<code>impact_paths</code>	The UI and the LLM prompts — this is the explainability payload. It shows <i>why</i> a file is considered affected, not just that it is.
<code>direct_dependencies</code>	The code generator, as candidate locations for a cross-file root-cause fix.
<code>crosses_service_boundary</code>	Risk signalling — a change escaping its top-level directory is flagged as higher-risk in review.
<code>confidence</code>	Reported as <code>high</code> when the traversal used a <code>CALLS</code> or <code>TESTS</code> edge (strong evidence), <code>medium</code> when only imports were involved, <code>low</code> when nothing was reachable.

A performance decision that matters. Collapsing graph edges into consumer→dependency maps is the expensive step, and the UI's bubble map calls `blast_radius()` once per file — which made large repositories quadratic. Because the graph is immutable after `build()`, those maps are computed once and cached, with the cache invalidated at the end of `build()`. Measured on a synthetic 150-file repository with 1,038 edges: **0.13s → 0.02s, roughly 6× faster, with byte-identical output.**

Agent catalogue

Ten agents, each with a single responsibility, a declared phase, and a typed return value. None of them calls another.

Agent	Phase	Responsibility and notable behaviour
Repo Mapper repo_mapper.py	1	Builds the knowledge graph, then converts every recorded syntax error into a HIGH-severity finding. Optionally compiles C/C++ files with <code>-fsyntax-only</code> concurrently via <code>asyncio.to_thread</code> , keeping only the first 3 errors per file since a cascade usually has one root cause. Also reports circular dependencies (capped at 10 cycles). No LLM calls.
Dependency Analyzer dependency_analyzer.py	1	Finds up to 10 requirements files (skipping <code>venv</code> / <code>node_modules</code>) and runs <code>pip-audit</code> , <code>safety</code> and <code>npm audit</code> concurrently, parsing JSON output into CVE findings with rule IDs. A missing CLI raises <code>FileNotFoundError</code> , which is caught and yields zero findings. No LLM calls.
Static Analysis static_analysis.py	1	Runs <code>bandit</code> and <code>npx eslint</code> concurrently. Notably, bandit's own <code>errors</code> array (files it could not parse) is turned into findings rather than dropped. ESLint messages are kept only at severity ≥ 2 , i.e. real errors, so they are treated as fixable rather than stylistic. No LLM calls.
Bug Hunter llm_bug_hunter.py	1	The primary detector. Ranks files by graph connectivity then size, then runs the two-stage Haiku→Sonnet funnel described in Section 9. Throttles between files to respect rate limits. A per-file LLM failure emits a warning event and continues to the next file.
Bug Investigation bug_investigation.py	2	For each finding: computes blast radius, traces data flow around the reported line, queries semantic memory for similar past fixes, and — only if the Bug Hunter did not already supply one — asks Sonnet for a root cause. That conditional saves one model call per finding.
Repair Planner repair_planner.py	3	Groups findings by epicenter file, builds a file-level dependency graph from the findings' affected-file sets, and orders work security-first then topologically within each bucket. Falls back to input order if the graph contains a cycle. No LLM calls.
Code Generation code_generation.py	4	Produces one anchored patch fixing every finding in a file, shown the epicenter plus graph neighbours under 20 KB and instructed to edit a neighbour when the root cause lives there. Rejects absolute and traversal paths before reading anything.
Validation validation_agent.py	4	Copies the repo to a temp directory, writes the patched content, then runs gates 1–3. Distinguishes "no test suite exists" from "tests failed" and reports which, rather than collapsing both into a failure.
Security Verification security_verification.py	4	Gate 4. Makes two copies — original and patched — scans both, and compares. Confirms the original rule is gone and no new medium-or-higher issue appeared. If bandit is absent it returns <code>gate_4_security_clean=True</code> but <code>security_scanner_available=False</code> , so the scorer withholds the 25%.
PR Author pr_author.py	5	Checks the critical-path detector, writes the PR body (Haiku summary plus issue list, confidence table, gate results, diff), then branches, commits, pushes and opens the PR as a draft when approval is required. A <code>finally</code> block always restores the worktree, so a failed attempt cannot leak writes into the next PR.

Deterministic vs probabilistic

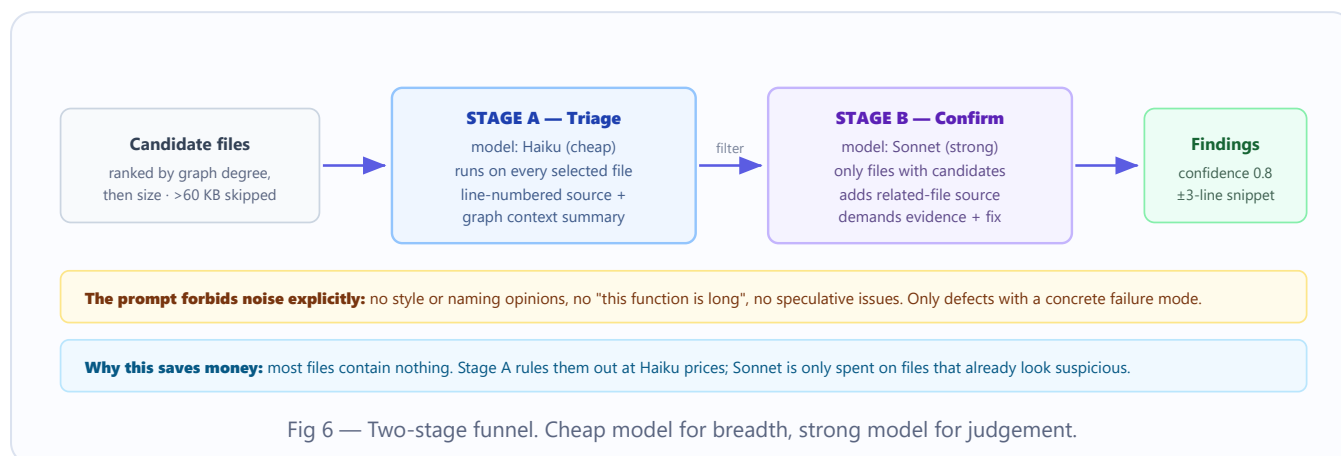
Six of the ten agents make no LLM calls at all. Detection, planning and validation are largely deterministic; the model is used where judgement is genuinely required — spotting a semantic bug, explaining a root cause, and writing the fix.

Consistent event protocol

Every agent emits `PipelineEvent` objects (`agent_start`, `agent_complete`, `agent_warning`, `agent_retry`) through an injected emitter. The UI renders whatever arrives, so adding an agent needs no UI change.

Detection strategy: scanners plus a two-stage LLM hunt

Static scanners are precise but narrow — bandit finds known insecure API usage, pip-audit finds known CVEs. Neither finds a wrong comparison operator or an off-by-one. The Bug Hunter covers that gap, and it does so in two stages for cost reasons.



From raw findings to a work list

The orchestrator merges all four detection sources, then applies three transformations before any fix is attempted:

1. **Path normalisation.** Scanners report absolute paths, Windows separators, and `./`-prefixed paths. All are converted to the graph's relative POSIX form, because a finding whose path does not match a graph node cannot be investigated.
2. **De-duplication, in two passes.** First on exact `(file, line, class, title)`, then on `(file, line, class)` for findings with a concrete line — because two tools describe the same defect with different wording. Deterministic tools are merged before the LLM hunter, so the precise description wins.
3. **Fixable vs report-only gating.** Security, functional and performance findings at or above the configured minimum severity become PR candidates. Code-quality findings are collected into a separate report-only list and shown in the UI without a fix being attempted.

Why gate instead of fixing everything. A PR that renames variables for readability costs a reviewer's attention and earns nothing. Restricting automation to defects with a concrete failure mode is what keeps the output worth reviewing.

Surgical patching: anchored SEARCH/REPLACE

The model never writes a file. It writes anchors, and the system decides whether they are valid.

Each edit is a pair: exact code that must already exist, and its replacement. The applier requires every anchor to match **exactly once**. Zero matches means the model invented or misquoted the context; more than one means the edit is ambiguous and could be applied in the wrong place. Both are refused with a diagnostic that becomes the retry prompt.

```
### FILE: backend/utils/money.py
<<<<<< SEARCH
    def apply_discount(total, pct):
        return total - pct
=====
    def apply_discount(total, pct):
        return total - (total * pct / 100)
>>>>>> REPLACE
```

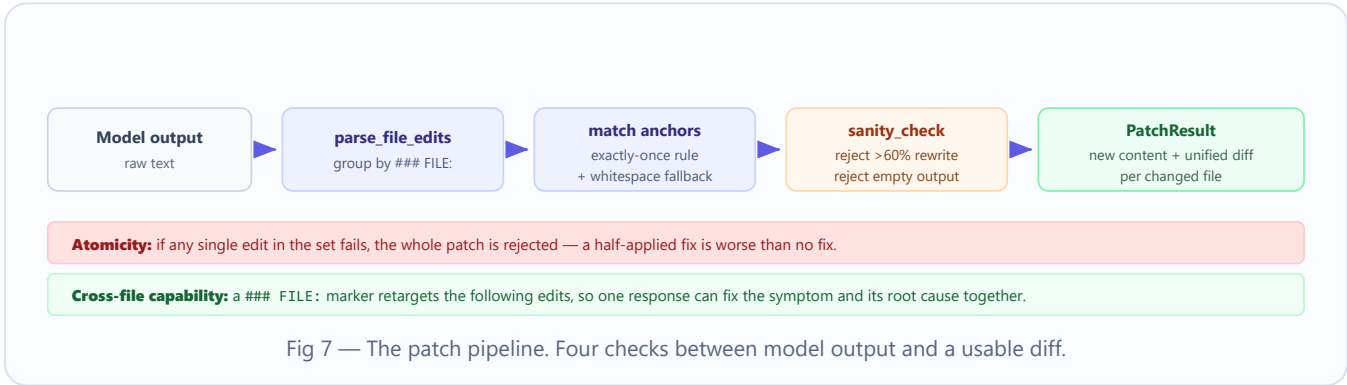


Fig 7 — The patch pipeline. Four checks between model output and a usable diff.

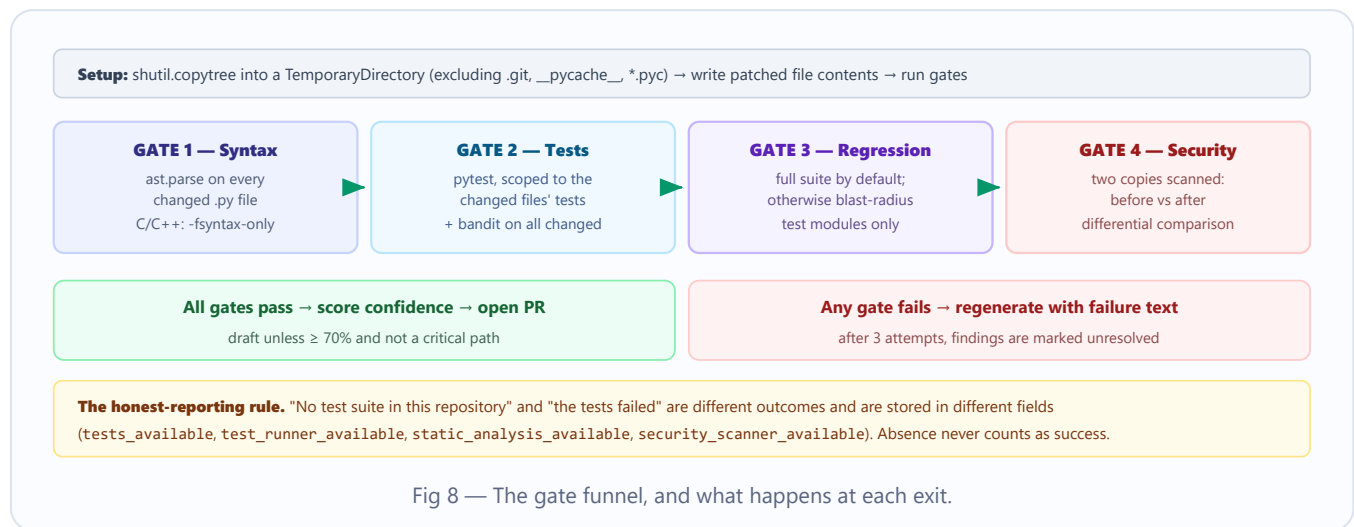
The three safeguards, and what each one prevents

Safeguard	Failure mode it prevents
Exactly-once matching	Hallucinated context, and edits landing on the wrong one of several similar code blocks. An ambiguous anchor produces an error telling the model to add surrounding context.
Whitespace-flexible fallback	Wasted retries. If an exact match fails, the applier retries line-by-line with trailing whitespace stripped — still requiring a unique match. This tolerates the single most common harmless model error without loosening the uniqueness rule.
60% rewrite ceiling	The whole-file-regeneration failure mode. If a "fix" changes more than 60% of a file over 25 lines, it is rejected as a rewrite in disguise, and an empty result is rejected outright.

Why this beats asking for a whole file. A whole-file response has to reproduce hundreds of correct lines to change three, and every reproduced line is an opportunity to drift. Anchored edits make the diff proportional to the fix, and make review proportional to the diff.

The four validation gates

Validation runs against a disposable copy of the repository, not the working tree. The patched content is written into the copy, and the gates run there — so a bad patch can never damage the checkout the PR will be cut from.



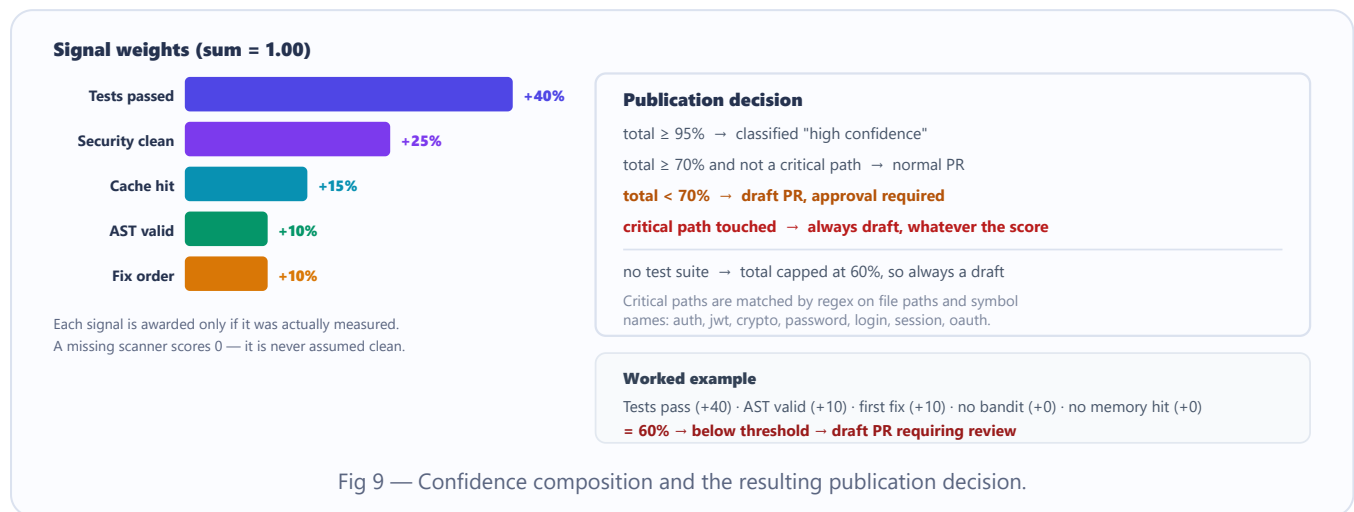
Gate details

Gate	Mechanism and the reasoning behind it
1 • Syntax	Every changed file must exist and parse. Python uses <code>ast.parse</code> ; C/C++ uses the system compiler's <code>-fsyntax-only</code> when one is available; other languages pass on existence alone. This is the cheapest possible check and it catches the most embarrassing failure — a patch that leaves the file unparseable.
2 • Tests	Finds test modules for every changed file by name (<code>test_<stem>.py</code> , <code><stem>_test.py</code>) anywhere in the repository, skipping vendored copies, and falls back to <code>-k</code> stem matching only if none are found by name. <code>pytest</code> exit code 5 ("no tests collected") is handled explicitly rather than being read as a failure. <code>Bandit</code> then scans all changed Python files — not just the primary one, because a cross-file patch can introduce a vulnerability in the root-cause file.
3 • Regression	By default runs the full suite, which is the honest check. When full-suite mode is disabled for expensive repositories, it falls back to the union of test modules reachable within the blast radius (default 2 hops) — using the graph to decide what "related" means instead of guessing.
4 • Security	Differential, not absolute. Two full copies of the repo are scanned, and the patch is accepted only if the originally-reported rule no longer fires <i>and</i> no new medium-or-higher issue appeared. An absolute scan would fail on pre-existing issues the patch never touched.

What this sandbox is and is not. It is a disposable filesystem copy that protects the working tree from a bad patch. It is **not** a security boundary — `pytest` from a cloned third-party repository executes as the current user, with no container, no resource limit and no network restriction. Real isolation would need Docker, and that is named as the top item of future work rather than implied to exist.

Confidence scoring and publication policy

Confidence is a weighted sum of five independently observed signals. It is deliberately simple arithmetic — no model produces this number, and it can be recomputed by hand from the gate results printed in the PR body.



Why these weights

Tests carry the most weight because a passing relevant test is the only signal that observes the code actually running. Security is second because a fix that introduces a vulnerability is worse than no fix. A cache hit is worth more than syntax validity because it means this class of bug was successfully fixed before in this repository, which is real evidence. Syntax validity is necessary but nearly free, so it earns little. Fix order is the weakest signal: the first fix in a topological ordering has the least chance of being invalidated by a later change, which is worth something but not much.

The 60% cap

Without a runnable test suite the 40% signal is unreachable, so the theoretical maximum is 60% — exactly the cap. Making the cap explicit means the policy holds even if the weights are later retuned, and it guarantees no untested repository ever produces a non-draft PR.

Why the system never merges

There is no merge call anywhere in the codebase. The highest-trust outcome is a normal (non-draft) pull request. Auto-merge would require a level of validation this project does not claim to provide, and saying so is more defensible than shipping it.

Semantic memory and cost control

Memory: an accelerator, never a dependency

After every successful fix the system stores a record in ChromaDB: the issue type, the root cause, the strategy that worked, the files changed, the confidence achieved and a validation summary. Later runs query it by semantic similarity, scoped to the repository, and a match above 0.85 cosine similarity becomes the `cache_hit` signal.

Two details matter. Record IDs are a deterministic SHA-256 of `(repo, bug type, file, root cause)` and writes use `upsert`, so re-running the pipeline updates a record instead of accumulating duplicates. And the entire `chromadb` import sits in a `try/except`, because it pulls native extensions that Windows application-control policies sometimes block.

If ChromaDB is unavailable, every memory method becomes a silent no-op: stores return, queries return an empty list, statistics return zero. The pipeline runs normally, simply without the 15% cache signal. Memory is an accelerator, so it was made optional by design.

Cost control

Mechanism	Effect
Haiku triage	Every file is screened by the cheap model; the strong model only sees files that already have candidates.
Conditional root cause	Investigation skips its Sonnet call when the Bug Hunter already produced a root cause — one saved call per finding.
Per-agent token budgets	Separate ceilings for code generation (8,192), investigation (4,096), hunting (4,096) and routing (1,024).
Size filters	Files over 60 KB are skipped by the hunter; related files over 20 KB are not included as patch context.
Findings grouped per file	One code-generation call fixes every finding in a file, rather than one call per finding.
Throttling	A configurable delay between files keeps the run inside provider rate limits rather than triggering retry storms.

Reliability of LLM calls

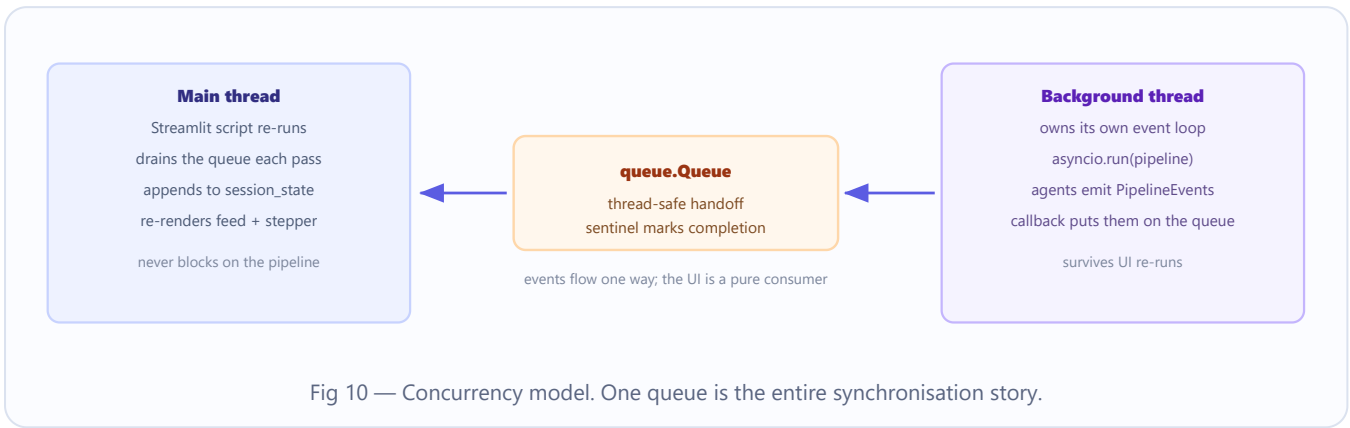
The client separates errors that retrying can fix from errors it cannot. A 400, 401, 403 or 404 raises immediately — retrying a bad API key wastes time and money. Everything else (rate limits, overload, timeouts, transient network faults) retries up to 4 times with exponential backoff plus random jitter, so concurrent agents do not resynchronise into a thundering herd.

Graceful degradation, summarised

Missing component	Guard in code	Consequence
ChromaDB	<code>try/except</code> around the import; <code>self.collection is None</code>	No cache signal; maximum confidence 85%.
bandit	<code>shutil.which("bandit") is None</code>	Gate 4 returns clean but flags the scanner unavailable; the 25% is withheld.
pytest	<code>importlib.util.find_spec("pytest")</code>	Gates 2 and 3 cannot award their signal; confidence capped.
pip-audit / safety / npm	<code>FileNotFoundError</code> caught per scanner	Zero dependency findings; the rest of discovery is unaffected.
ESLint	<code>FileNotFoundError</code> on <code>npx</code>	JavaScript files are covered by the LLM hunter only.
C/C++ compiler	<code>find_cpp_compiler()</code>	C/C++ syntax checking is skipped; Python analysis is unaffected.
GitHub token	<code>if not settings.github_token</code>	Everything runs; the PR is fully prepared and displayed but not pushed.

User interface architecture

The front end is one Streamlit file. The interesting problem it solves is that Streamlit re-runs the whole script on every interaction, while the pipeline is a long-running async job that must stream progress.



Why Streamlit replaced a FastAPI + SSE design

The earlier design needed a server, an async event-stream endpoint, a separate frontend and CORS handling — four moving parts to display a list of events. A background thread and a queue deliver the same live feed inside one process. For a single-user analysis tool this is the right trade: less surface area, nothing to deploy, and the pipeline stays independent of the UI.

The repository bubble map

A vis-network force-directed view of the graph, where bubble size reflects blast radius and colour reflects service boundary. It is bounded on purpose — 400 files, 1,500 edges, and hub nodes that would generate cliques larger than 25 are skipped — because an unbounded force layout on a large repository will freeze the browser.

What the interface shows

View	Contents
Setup guide	Connection status for the API key, base URL and GitHub token, rendered before a run can start.
Live activity feed	Per-agent event cards with phase colouring, a stepper bound to <code>JobStatus</code> , and running counts of elapsed time, events, findings and PRs.
Findings explorer	Findings grouped and filterable by severity, issue class and source tool, each with its blast radius, impact paths and code snippet.
Repair plan	The ordered work list — which file is fixed first and why, with the pre-score and dependency list per item.
Pull requests	Title, confidence breakdown, gate results, blast-radius summary, root cause and the full diff; draft status shown explicitly.
Repository map	The interactive bubble graph, which can be built on its own without running the full pipeline.

Engineering quality: tests and defects fixed

The project carries 11 automated tests covering the parts where a silent regression would be most damaging: graph construction and impact direction, patch application, confidence arithmetic, memory behaviour, and UI helpers. They run without network access — the test fixture injects a dummy API key precisely so that no test can make a real model call.

Test module	What it protects
<code>test_knowledge_graph.py</code>	Node and edge construction, and that impact analysis traverses in the correct direction.
<code>test_patching.py</code>	Anchor matching, ambiguity rejection, and the whole-file-rewrite guard.
<code>test_confidence.py</code>	Weight arithmetic and the no-tests cap.
<code>test_memory.py</code>	Store/query behaviour, including the no-op path when ChromaDB is absent.
<code>test_ui_helpers.py</code>	Formatting helpers used by the dashboard.

Defects found and fixed during review

A systematic read of the codebase produced five defects, each verified before and after the fix. They are listed here because the reasoning behind them is a fair summary of what the project's own analysis is looking for.

#	Defect	Why it mattered, and the verification
1	Credential leak in error handling <code>utils/github_client.py</code>	The GitHub token is embedded in the clone URL. Both <code>CalledProcessError</code> and <code>TimeoutExpired</code> place the full command in their message — which the UI prints in its error box and traceback expander. Now re-raised as a clean <code>RuntimeError</code> with <code>from None</code> so the original never propagates. Verified: token present in the error before the fix, absent after.
2	<code>rstrip("./")</code> corrupting dotfile paths <code>orchestrator.py</code> , <code>code_generation.py</code>	<code>str.rstrip</code> strips a <i>character set</i> , not a prefix. So <code>".env"</code> became <code>"env"</code> and <code>".github/workflows/ci.yml"</code> became <code>"github/workflows/ci.yml"</code> — paths that match no graph node, so findings in those files could never be investigated or patched. Replaced with <code>removeprefix("./")</code> . Verified against all three cases.
3	Unguarded <code>stat()</code> during graph build <code>knowledge_graph.py</code>	A broken symlink or unreadable file raised <code>OSError</code> mid-walk and aborted the entire repository scan. Now caught per file, skipping that one entry. Verified: the scan completes and continues indexing.
4	Quadratic blast-radius computation <code>knowledge_graph.py</code>	The consumer→dependency maps were rebuilt on every call, and the bubble map calls <code>blast_radius()</code> once per file. Cached, with invalidation at the end of <code>build()</code> . Verified on 150 files / 1,038 edges: 0.13s → 0.02s, output byte-identical.
5	Security gate scanning only the epicenter <code>validation_agent.py</code>	Gate 2's bandit scan covered only the primary file, so a cross-file patch could introduce a vulnerability in the root-cause file and still pass. Now scans every changed Python file. Verified: a patch introducing <code>subprocess.call(cmd, shell=True)</code> in a secondary file passed the old gate and is rejected by the new one (bandit B602).

Result: 11 of 11 tests pass before and after all five changes. The performance fix was confirmed to produce identical output, so the speed-up carries no behavioural risk.

Limitations, trade-offs and future work

Stating these plainly is part of the design. Each one is a conscious trade, not an oversight.

Limitation	Assessment and the path forward
The sandbox is not isolation	Validation copies the repository to a temp directory, but the tests inside it run as the current user with full filesystem and network access. Cloning a hostile repository and running its test suite is genuinely dangerous. Next step: execute gates inside a Docker container with no network and a read-only mount. This is the single highest-value improvement.
Phase 4 is serial	Fixes are generated and validated one file at a time, so wall-clock time grows linearly with the number of files to fix, and the run is bound by model latency. This is a correctness-driven choice — one shared worktree — but it is the main scalability ceiling. Next step: one git worktree per fix, enabling a bounded worker pool.
Deep Python bias	AST parsing, symbol indexing, call resolution and test discovery are Python-specific. JavaScript and C/C++ get file-level edges and LLM analysis but no symbol graph, so cross-file impact in those languages is coarser. Next step: tree-sitter grammars to bring other languages up to the same fidelity.
Framework detection is regex-based	API routes, ORM queries and config access are matched with regular expressions, so an unusual decorator or a dynamically-registered route is missed. The cost of a miss is one absent edge rather than an incorrect one, which is why the trade was accepted.
Broad exception handling	Optional subsystems catch <code>Exception</code> and continue, which is what makes graceful degradation possible but also hides unexpected failures. Next step: narrow each handler to the specific exception it is designed for and log the rest.
No cap on the number of PRs	<code>max_files_to_fix</code> defaults to 0, meaning unlimited. On a repository with many findings this produces many pull requests and a large model bill. A sensible default cap is a one-line change.
Cost is unmetered	The <code>PipelineJob</code> model carries a <code>cost_estimate_usd</code> field that is never populated. Token accounting per phase would make the two-tier strategy's cost/benefit measurable rather than argued.

Conclusion

The system demonstrates that autonomous repair becomes tractable when the problem is decomposed correctly: a deterministic structural layer that knows how the repository fits together, a probabilistic layer that supplies judgement where judgement is required, and a verification layer that refuses to let unverified output through.

The design decisions that carry the most weight are the ones that constrain the model rather than trust it. Anchored edits make hallucinated context unusable. Atomicity makes partial application impossible. Differential security scanning makes "did this patch make things worse" a measurable question. And the confidence model's refusal to award points for signals it could not measure is what keeps the number meaningful — a 60% score genuinely means less was verified, not that the fix is probably wrong.

What the project does not do is equally deliberate. It does not merge. It does not claim isolation it has not implemented. It does not treat a missing scanner as a passing scanner. Those boundaries are what make the parts it does claim believable.

29

Python modules

~4,900 lines across four layers

11 / 11

Tests passing

graph, patching, scoring, memory, UI

5

Defects fixed

each verified before and after

One-sentence summary. An explainable multi-agent pipeline that combines a directional knowledge graph, two-tier LLM analysis, anchored patch generation, four-gate sandbox validation and GitHub automation — built so that every claim it makes about a fix can be traced back to something it actually measured.